

Ciencias de la Computación II

Optimización de la Planificación Académica mediante Teoría de Grafos

Documento Consolidado de Especificación y Análisis Algorítmico

Autores

Nombre: Sergio Nicolás Osorio Guevara **Código:** 20241020073

Nombre: Silvana Martínez Pardo **Código:** 20241020010

Nombre: Arley Santiago Alvarez Ortiz **Código:** 20241020008

Nombre: Laura Sofía Cuadros Niño **Código:** 20222020160

Nombre: Emmanuel Guerrero Piza **Código:** 20202020039

Institución: Universidad Distrital Francisco José de Caldas

Proyecto Curricular: Ingeniería de Sistemas

Asignatura: Ciencias de la Computación II

Índice

Parte I: Abstracción del Problema	2
1. El Problema	2
2. ¿Cómo usamos los grafos?	2
2.1. Fase 1: El Grafo de los Profesores (¿Quién dicta la clase?)	3
2.2. Fase 2: El Grafo de los Conflictos (¿A qué hora es la clase?)	3
3. Resultados en Pruebas Reales	4
3.1. El Repitente Salvado	4
3.2. El Estudiante Ambicioso	5
3.3. El Colapso Controlado (Alerta de Decanatura)	5
Parte II: Traducción al Lenguaje Matemático y Técnico	6
4. Definición Formal del Problema	6
5. Fase 1: Grafo Bipartito de Asignación Docente	6
5.1. Condición Lógica de Adyacencia (Generación de Aristas)	6
5.2. Análisis de Complejidad de Validación	7
5.3. Algoritmo de Resolución: Emparejamiento Máximo Bipartito	7
6. Fase 2: Grafo No Dirigido de Conflictos Temporales	7
6.1. Generación de la Matriz de Adyacencia (Patrón Strategy)	8
6.2. Algoritmo de Resolución: Coloración DSatur	8
7. Análisis de Límites y Arquitectura Fail-Fast	8
7.1. Generación de Cliques	9
7.2. Excedente de Hardware/Infraestructura	9
8. Conclusiones y Trabajo Futuro	9
8.1. Conclusiones	9
8.2. Trabajo Futuro	9

Parte I: Abstracción del Problema

¿Qué es el University Timetabling Problem?

Cada semestre, los coordinadores académicos se enfrentan a un rompecabezas colosal: armar el horario de clases. En Ciencias de la Computación, esto se conoce como el **University Timetabling Problem (UTP)**. Es un problema tan complejo que resolverlo a mano resulta en errores, cruces de horarios y estudiantes insatisfechos.

1. El Problema

Para que este sistema sea útil en el mundo real (Facultad de Ingeniería), no basta con asignar horas al azar. Se deben respetar cuatro reglas fundamentales:

1. **Idoneidad docente (¿Quién dicta qué?):** Un profesor experto en “Ciencias Básicas” no debe dictar “Ingeniería de Software”. El sistema debe asegurar que el docente domina el área asignada.
2. **Protección al estudiante repitente (vetos):** Si un estudiante reprobó una materia con un profesor específico, el sistema tiene “vetado” volver a cruzar sus caminos. Esto garantiza equidad en la evaluación y previene la deserción.
3. **El límite físico de la universidad:** La facultad no tiene salones ni tiempo infinito. Se definió un límite de **36 franjas** horarias a la semana (Lunes a Sábado, 6 clases por día). El sistema no puede inventar horarios.
4. **La realidad de la malla curricular:** Muchos estudiantes adelantan o atrasan materias. Si alguien inscribe “Cálculo Integral” (Nivel 2) y “Bases de Datos” (Nivel 4), el sistema debe garantizar que ambas clases no se dicten a la misma hora para ese estudiante.

2. ¿Cómo usamos los grafos?

Para que una computadora resuelva esto, traducimos el problema a la **Teoría de Grafos** (puntos conectados por líneas). Dividimos el problema en dos fases:

2.1. Fase 1: El Grafo de los Profesores (¿Quién dicta la clase?)

Imaginemos dos columnas: a la izquierda todos los profesores y a la derecha todas las clases. ¿Cuándo dibujamos una línea que conecte a un profesor con una clase? Solo si:

- El profesor sabe del tema.
- Ningún estudiante de esa clase ha reprobado antes con ese profesor.

Para lograrlo, el sistema busca al profesor perfecto. Si la primera opción ya está ocupada, el sistema usa un algoritmo llamado **DFS (Búsqueda en Profundidad)**. Este algoritmo no es egoísta: si una clase necesita a la profesora Isabel, pero ella está ocupada, el sistema le pregunta al otro grupo: “¿Podemos cambiar a Isabel por el Profesor Carlos?”. Si se puede, Carlos toma el otro grupo, Isabel se libera y toma el nuevo. Esto se llama buscar **Caminos Aumentantes**.

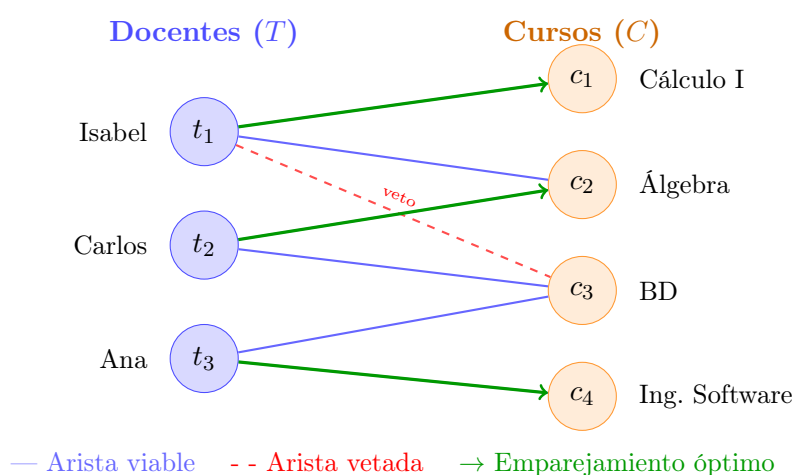


Figura 1: Grafo bipartito $G_1 = (T, C, E_1)$. Las aristas verdes representan el emparejamiento máximo obtenido; la arista roja discontinua indica una asignación bloqueada por veto de repetencia.

2.2. Fase 2: El Grafo de los Conflictos (¿A qué hora es la clase?)

Una vez que cada clase tiene profesor, armamos el horario. Aquí los “puntos” son las clases y las “líneas” significan **PELIGRO**: estas dos clases no pueden ocurrir a la misma hora.

Dibujamos líneas de peligro si:

- Son de la misma materia o las dicta el mismo profesor.
- Son del mismo semestre o comparten al menos un estudiante matriculado.

Para resolver esto, usamos un algoritmo de coloreado llamado **DSatur**. Imaginemos que cada “color” es una franja horaria (Ej. Azul = Lunes 6 AM). DSatur es muy inteligente: busca primero la clase más “saturada” (la que tiene más estudiantes cruzados

y más conflictos) y le asigna un horario primero. Atacar los cuellos de botella desde el principio permite comprimir todo el horario usando la menor cantidad de salones posible.

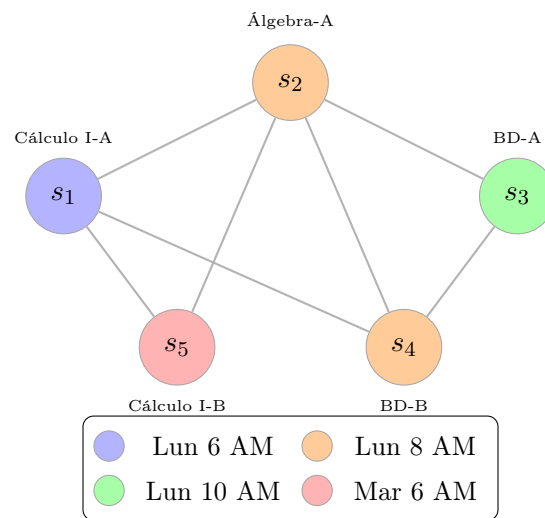


Figura 2: Grafo de conflictos $G_2 = (S, E_2)$. Cada color representa una franja horaria distinta asignada por DSatur. Nodos del mismo color no comparten arista, garantizando cero colisiones.

3. Resultados en Pruebas Reales

Configuración del banco de pruebas

Sometimos el sistema a **100 estudiantes** cursando materias simultáneas, **10 profesores** y **40 historiales de reprobación**.

3.1. El Repitente Salvado

Caso 1 — Protección por veto docente

El sistema detectó que un estudiante vería clase con el profesor que lo reprobó. En milisegundos, reconfiguró toda la planta docente del área para darle un nuevo profesor **sin afectar al resto de la facultad**.

3.2. El Estudiante Ambicioso

Caso 2 — Múltiples semestres simultáneos

Un alumno inscribió materias de **3 semestres distintos**. DSatur detectó el peligro y separó todas sus clases en horarios diferentes. **Cero colisiones**.

3.3. El Colapso Controlado (Alerta de Decanatura)

Caso 3 — Comportamiento Fail-Safe

Forzamos el sistema hasta que matemáticamente necesitaba **40 franjas horarias**, pero el límite de la universidad era 36. En lugar de colapsar o inventar clases en domingo, el sistema se detuvo en la franja 36 y emitió una “**Alerta de Decanatura**”. Esto demuestra que el software sirve como herramienta administrativa para avisar cuándo hay que construir más salones o contratar más docentes.

Parte II: Traducción al Lenguaje Matemático y Técnico

Objetivo de esta sección

Esta sección formaliza las lógicas explicadas anteriormente utilizando notación matemática, teoría de conjuntos, análisis de grafos y complejidad algorítmica.

4. Definición Formal del Problema

El *University Timetabling Problem* es abordado como un problema de **optimización combinatoria NP-Hard**. Se define el universo de restricciones mediante tuplas, donde el objetivo es encontrar:

Funciones de Asignación

- Una **función de asignación docente** $f : C \rightarrow T$
- Una **función de asignación temporal** $h : S \rightarrow H$

Minimizando colisiones sujetas a *Hard Constraints* estrictas.

5. Fase 1: Grafo Bipartito de Asignación Docente

Sea el grafo no dirigido bipartito $G_1 = (T, C, E_1)$, donde:

- $T = \{t_1, t_2, \dots, t_n\}$ es el conjunto de docentes disponibles.
- $C = \{c_1, c_2, \dots, c_m\}$ es el conjunto de cursos/grupos requeridos.
- $E_1 \subseteq T \times C$ es el conjunto de aristas (asignaciones viables).

5.1. Condición Lógica de Adyacencia (Generación de Aristas)

Definición de Adyacencia

Sean:

- $Area(t_i)$: el conjunto de áreas de conocimiento del docente t_i .
- $Req(c_j)$: el área requerida por la materia c_j .

- $Est(c_j)$: el conjunto de estudiantes matriculados en el grupo c_j .
- $Vet(t_i)$: el conjunto de estudiantes que tienen vetado al docente t_i por repitencia.

Una arista $(t_i, c_j) \in E_1$ existe si y solo si:

$$(t_i, c_j) \in E_1 \iff [Req(c_j) \in Area(t_i)] \wedge [Est(c_j) \cap Vet(t_i) = \emptyset]$$

5.2. Análisis de Complejidad de Validación

La intersección de conjuntos se implementó mediante **Hash Sets** (`Set.intersection()`), reduciendo la complejidad de búsqueda de $\mathcal{O}(|Est| \times |Vet|)$ a un caso promedio de:

$$\mathcal{O}(\min(|Est|, |Vet|))$$

5.3. Algoritmo de Resolución: Emparejamiento Máximo Bipartito

Para encontrar la inyección óptima de profesores a cursos, se aplicó un algoritmo de **Emparejamiento Máximo Bipartito mediante Caminos Aumentantes (DFS)**.

¿Por qué no Greedy?

En lugar de asignaciones *Greedy* que estancan el sistema en óptimos locales, el algoritmo expande rutas alternativas recursivamente, garantizando la solución global óptima.

Complejidad Temporal: $\mathcal{O}(|T| \cdot |E_1|)$

6. Fase 2: Grafo No Dirigido de Conflictos Temporales

La asignación de horarios se modeló como un **problema de coloración de vértices**. Se define el grafo de conflictos $G_2 = (S, E_2)$, donde:

- $S = \{s_1, s_2, \dots, s_k\}$ representa las sesiones individuales. Un curso de 3 créditos expande 3 vértices disjuntos en S .
- $E_2 \subseteq S \times S$ representa los conflictos temporales.

6.1. Generación de la Matriz de Adyacencia (Patrón Strategy)

Condiciones de Conflicto

Una arista $(s_u, s_v) \in E_2$ existe si se cumple **al menos una** de las siguientes proposiciones lógicas:

1. $Curso(s_u) = Curso(s_v)$ *(Misma materia)*
2. $f(Curso(s_u)) = f(Curso(s_v))$ *(Mismo docente asignado en la Fase 1)*
3. $Nivel(s_u) = Nivel(s_v)$ *(Mismo semestre de la malla curricular)*
4. $Est(s_u) \cap Est(s_v) \neq \emptyset$ *(Intersección no nula de estudiantes)*

6.2. Algoritmo de Resolución: Coloración DSatur

El objetivo es encontrar el **Número Cromático** $\chi(G_2)$, equivalente a la cantidad mínima de franjas horarias necesarias, tal que:

$$\chi(G_2) \leq H_{max} \quad \text{donde } H_{max} = 36$$

Se implementó **DSatur (Degree of Saturation)**:

1. Se define el **grado de saturación** $Sat(s_i)$ como la cardinalidad del conjunto de colores asignados a la vecindad $N(s_i)$.
2. En cada iteración de $\mathcal{O}(|S|)$, se selecciona el vértice:

$$\arg \max_{s_i \in S} Sat(s_i)$$

En caso de empate, se desempata por el vértice con mayor grado estático $deg(s_i)$.

3. Se asigna el menor color viable $c \geq 1$.

DSatur vs. Welsh-Powell

A diferencia de *Welsh-Powell*, que evalúa un ordenamiento topológico estático, DSatur evalúa la densidad dinámica del grafo de forma **adaptativa**, abordando primero los “cliques” (estudiantes con alta densidad de cruces).

7. Análisis de Límites y Arquitectura Fail-Fast

Durante las pruebas de estrés, el comportamiento del sistema fue validado matemáticamente:

7.1. Generación de Cliques

Ante estudiantes adelantando múltiples materias simultáneas, se formaron **subgrafos completos** $K_n \subseteq G_2$. El algoritmo garantizó correctamente que para cualquier clique K_n , se requieren n colores distintos, logrando 0 % de colisiones.

7.2. Excedente de Hardware/Infraestructura

Comportamiento Fail-Safe validado

Cuando el algoritmo iterativo de DSatur determinó que $\chi(G_2) = 40$ y dado que la restricción de infraestructura era estricta ($H_{max} = 36$), el sistema actuó bajo un **patrón Fail-Safe**. Desencadenó una excepción controlada, rechazando la asignación de colores $c > 36$. Esto provee a la administración evidencia algorítmica para la toma de decisiones sobre expansión de infraestructura física.

8. Conclusiones y Trabajo Futuro

8.1. Conclusiones

La implementación de arquitectura desacoplada en dos estructuras de grafos ortogonales:

- **Grafo Bipartito** — para la gestión de Recursos Humanos (asignación docente).
- **Grafo Simple** — para la gestión del Cronograma (asignación de franjas horarias).

demostró ser una solución robusta y matemáticamente adecuada para el UTP, además, la separación del problema en los dos grafos permitió resolver de forma independiente y eficiente cada dimensión del problema, cumpliendo con la totalidad de las restricciones definidas en el proyecto. Además se determinó que los algoritmos seleccionados no solo son correctos sino también resilientes ante escenarios límite, transformando una falla potencial en una herramienta de diagnóstico institucional.

8.2. Trabajo Futuro

Línea de investigación propuesta

Como trabajo futuro se propone someter el vector de colores resultante de DSatur a algoritmos de **optimización metaheurística**, como el **Recocido Simulado (Simulated Annealing)**, tomando como función objetivo la minimización de la varianza temporal intradiaria en el horario de cada estudiante (reducción de “huecos”).